```
wait = (duration, callback) ->
  setTimeout callback, duration

wait 5, ->
  turtle
    .forward 10
    .forward 3
```

A dot on a newline does not close the callback function passed to wait.

In CoffeeScript 1.7 the function call and fluent chain compile as follows:

```
wait(5, function() {
  return turtle.forward(1).rotate(90);
});
```

Compiled JavaScript.

Now, back to Scruffy's API work and how a fluent interface makes life easier for him.

7.6.3. Creating fluent interfaces

If you're creating an API from scratch, then a fluent interface requires a specific usage of `this` (a.k.a. `@`). Remember, inside a method call, `@` refers to the current object (the receiver of the method call). Consider what it means to use `@` as the final value in a method:

```
class Turtle
  rotate: (degrees) ->
    # rotate the turtle
    @
```

It doesn't matter how the turtle is rotated.

Return the turtle.

Because `@` on a line by itself at the end of a function looks a bit lonely, you'll often see the `this` keyword used instead:

```
class Turtle
  rotate: (degrees) ->
    # rotate the turtle
    this
```

Using `this` as the final value in a method returns the object that received the method call. So, if you return `this`, then method calls can be chained: